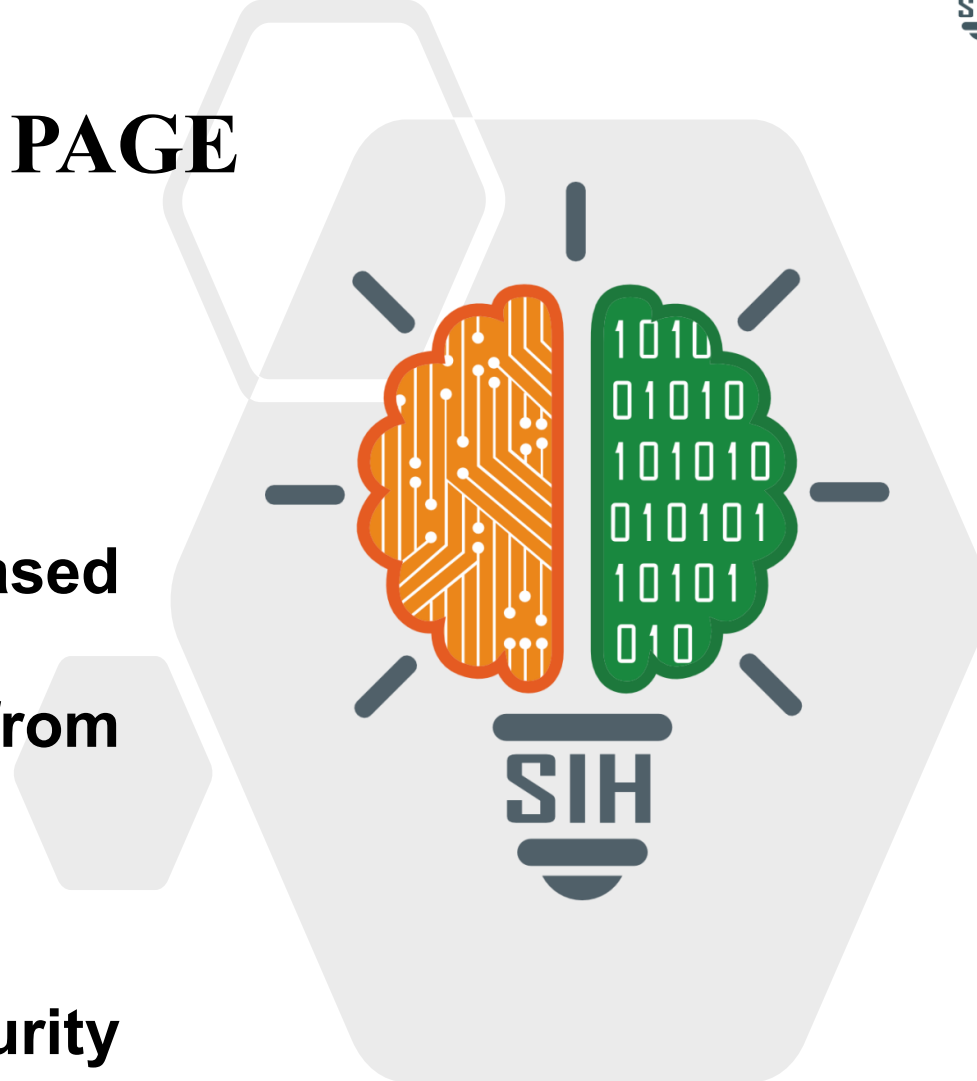


TITLE PAGE

- Problem Statement ID: SIH26153
- Problem Statement Title: AI-based
Network Attack Forecasting from
Network Traffic Data
- Theme: Blockchain & Cybersecurity
(Organization: NTRO)



❖ Proposed Solution

- A temporal attack forecaster: reads 10 x 30-second network-state windows (18 flow features) and forecasts attack probability for each of the next 5 windows (2.5 min ahead), plus the coming MITRE ATT&CK stage
- Working prototype today: FastAPI + Next.js console, fully offline on one laptop — historical attack scenarios AND a live packet sensor (Npcap/scapy) feed the same trained model
- Every warning is explained: per-feature attribution (Integrated Gradients) + transparent ATT&CK stage rules

How it addresses the problem

- Implements the PS's world-model requirement directly — $P(S_{t+1} | S_t)$ learned from telemetry and rolled forward $K=5$ steps; not a static per-flow classifier
- Beats the required logistic baseline ~2x on PR-AUC (0.656 vs 0.333) on identical features and split
- Explainability (PS asks for SHAP/attention): every prediction ships with feature attribution and an independent rule-based stage cross-check

Innovation and uniqueness

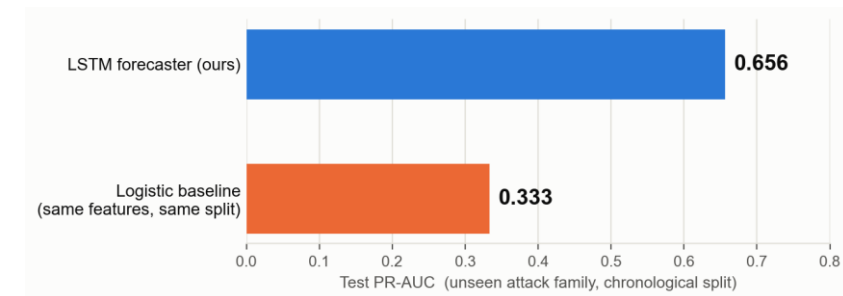
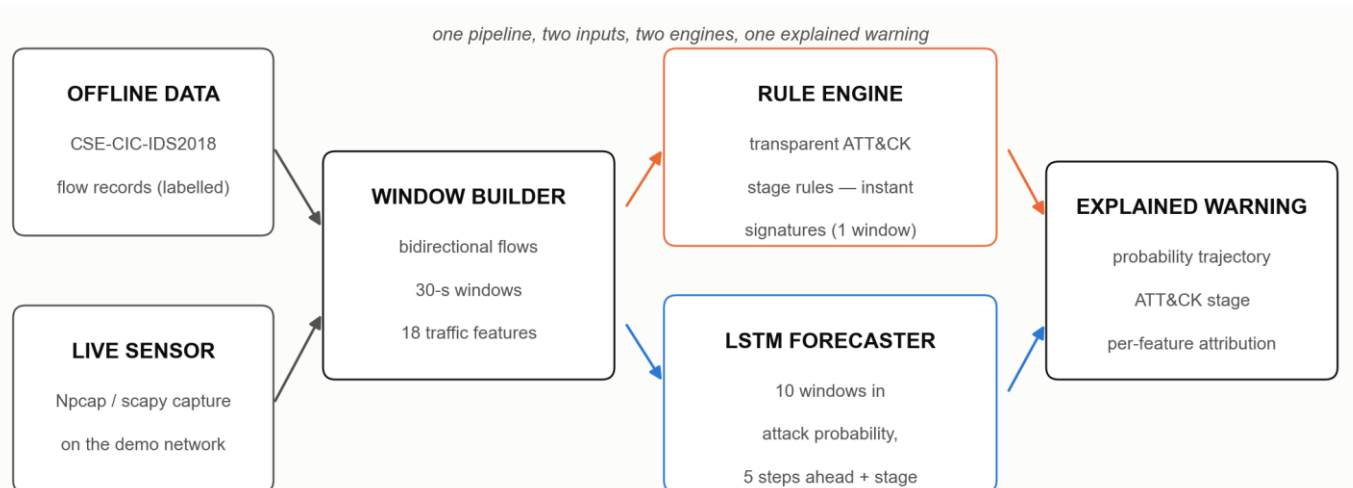
- Forecast, don't just detect: probability trajectory + kill-chain stage trajectory = analyst decision support while the attack is still unfolding
- Two engines by design: transparent rules catch instant signatures (recon scan flagged in one window); the LSTM forecasts sustained progression — both verified live over real Wi-Fi
- 57K parameters · 0.23 MB · 2.6 ms CPU inference → edge-deployable, offline, flow-metadata only (no payloads — privacy-preserving)

Technologies to be used

- Python · PyTorch (2-layer LSTM, multi-task heads) · scapy/Npcap live capture · FastAPI backend · Next.js + TypeScript console · CSE-CIC-IDS2018 benchmark (~6,200 windows, 1,486 attack)

Methodology and process for implementation

- Flow records → 30-s windows x 18 features → chronological 70/15/15 split with boundary purge (no leakage; test split = an attack family absent from training)
- log1p + standardize fitted on train only — one shared transform for baseline, LSTM and the live sensor
- 2-layer LSTM (hidden 64): 5 progression logits + 6-stage head; alert threshold from a 5% false-positive budget on validation
- Independent rule engine (6 ATT&CK rules) validated against labelled windows; Integrated Gradients attribution on every prediction



FEASIBILITY AND VIABILITY



- **Analysis of the feasibility of the idea**

- Already built and verified end-to-end: offline benchmark + live sensor demo; in rehearsal a real attack over Wi-Fi crossed the alert threshold during a sustained UDP sweep while benign traffic stayed LOW
- 0.23 MB model, 2.6 ms CPU inference, fully offline — runs on a commodity laptop, no cloud dependency

Potential challenges and risks

- Benchmark ↔ production traffic shift (benchmark flows are long-lived aggregates; production sees short transactions)
- Class imbalance and rare attack families (~24% of windows contain attack)
- False alarms erode analyst trust; no system can warn before the first attack packet exists

Strategies for overcoming these challenges

- Live input conditioned to the model's validated training domain + rule engine on raw features; NetFlow/IPFIX named as the production input
- PR-AUC focus, chronological split, high-precision operating point (FPR 0.6%); cross-family test split measures transfer honestly
- Every alert explained (attribution + rules); positioned as decision support, not auto-blocking; limitations stated up front
- Scale path: windowing at each sensor/tap, one shared 0.23 MB model centrally or at the edge

IMPACT AND BENEFITS



- **Potential impact on the target audience**

- SOC analysts, CERTs and agencies (NTRO context), critical-infrastructure operators — anyone with flow telemetry and alert fatigue
- Early, explained warnings redirect analyst attention to the right flows while an attack is still unfolding

Benefits of the solution

- Economic: breaches average US\$4.44M and ~241 days to identify and contain (IBM 2025); early warning + explanation shortens both; security AI/automation is associated with ~\$1.9M lower breach costs
- Social: protects critical infrastructure and public services; metadata-only analysis preserves privacy (no packet payloads)
- Analyst welfare: explained, high-precision warnings instead of black-box alert floods
- Environmental: software-only, 57K-parameter model — no new hardware; edge deployment minimises data movement
- Scalable and globally comparable: trained/evaluated on the international benchmark dataset; the architecture (sensor → windows → model) is network-agnostic — any organization exporting flow data can deploy it

RESEARCH AND REFERENCES



- **Research and references**

- CSE-CIC-IDS2018 dataset — Canadian Institute for Cybersecurity, UNB: <https://www.unb.ca/cic/datasets/ids-2018.html>
- MITRE ATT&CK framework: <https://attack.mitre.org>
- Sundararajan, Taly & Yan (2017), 'Axiomatic Attribution for Deep Networks' (Integrated Gradients), ICML
- Hochreiter & Schmidhuber (1997), 'Long Short-Term Memory', Neural Computation
- IBM Security, 'Cost of a Data Breach Report 2025': <https://www.ibm.com/reports/data-breach>
- Open-source stack: Python, PyTorch, scapy, FastAPI, Next.js
- Project repository (working prototype + code): <add GitHub link before submission>